

æSIP: μ Arch-aware ASIP-ISA Co-Design via Program Synthesis, Equality Saturation, and External Don't Cares

Abstract—An increasing number of applications, including implantables, IoT devices, and printed electronics, impose stringent power and area constraints. With the end of Dennard scaling, Application Specific Integrated Processors (ASIPs) have emerged as a promising solution, reducing power consumption and silicon area by sacrificing generality compared to general-purpose processors. However, existing approaches predominantly optimize hardware through software profiling, potentially overlooking optimization opportunities through software rewriting. We propose æSIP, a hardware-software co-optimization framework for efficient ASIP design. Our framework leverages e-graph data structures to explore semantically equivalent software implementations through rewriting rules derived from program synthesis. We employ a divide-and-conquer approach that performs local saturation at basic-block granularity followed by Integer Linear Programming (ILP)-based global extraction at whole-program scope, enabling scalability to real-world applications. We develop a don't care-based hardware optimizer that automatically generates ASIP designs for each equivalent program variant, enabling agile design space exploration. We further incorporate a clustering algorithm that groups applications with similar rewrite/ISA characteristics, along with a constraint-based sharing mechanism that limits the number of ASIP variants, thereby enabling efficient reuse across workloads and balancing area/power efficiency with NRE cost.

To our knowledge, this represents the first systematic hardware–software co-design framework for agile ASIP development. Experimental evaluation on widely used embedded benchmark suites, MiBench and Embench, demonstrates up to 29.7% area reduction and 8.3% power savings compared to state-of-the-art ASIP generators. Moreover, æSIP exploits generalization opportunities across diverse workloads without compromising specialization. Our trade-off analysis shows that using only four shared ASIPs, æSIP achieves a 14.17% area reduction with just 0.23% latency overhead, whereas even fully per-application specialization attains only a 19.11% area reduction at the same latency target.

I. INTRODUCTION

The proliferation of embedded systems has intensified the demand for energy-efficient, area-constrained processing solutions. For instance, flexible and printed electronics, characterized by minimal transistor budgets and negligible mask costs, make ultra-optimized, application-specific designs economically compelling [9], [10], [40]. Furthermore, emerging applications such as microprocessors for edge artificial intelligence [6], [40], controllers for cryogenic quantum computing systems [7], and brain-computer interfaces have diversified computational requirements while imposing stringent constraints [4]. The reduction of area and power consumption in processors enables increased computational capacity while

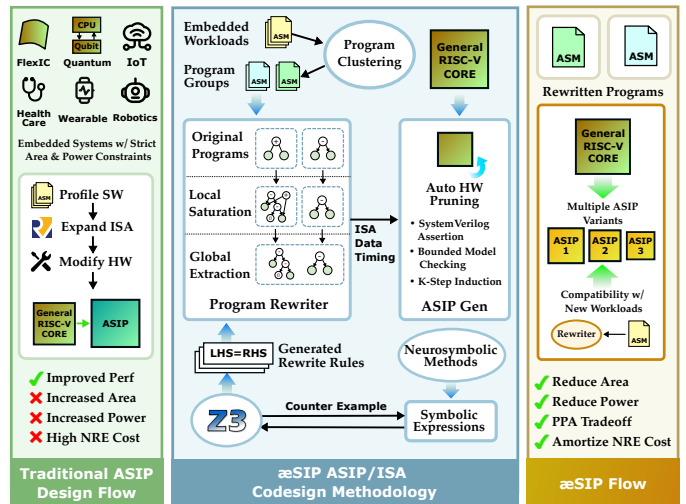


Fig. 1. α SIP: an agile hardware-software co-design framework. Traditional ASIP [48] design profiles target applications and designs specialized hardware. Despite its performance benefits, it falls short for embedded systems with strict area/power constraints. RISPs address this by pruning unused hardware according to actual instruction usage. α SIP enhances this approach through hardware-aware program rewriting. By combining neurosymbolic-based program synthesis, divide-and-conquer equality saturation, and external don't care-based hardware pruning, α SIP achieves end-to-end ASIP design tailored for target applications with up to 40% area reduction under given latency constraints.

minimizing thermal disturbances, thereby enhancing overall system performance and reliability [22], [55], [61].

In these resource-constrained environments, designers must navigate not only tight area and power budgets, but also increasingly diverse application requirements. This has driven growing interest in application-specific instruction-set processors (ASIPs), which trade full generality for domain-tailored ISA and microarchitectures: depending on the target, specialization may aim to reduce area and energy, lower latency, or even accept modest resource overheads in exchange for critical domain-specific functionality [26], [31].

Historically, ASIP design has followed an additive methodology, augmenting base instruction sets with customized extensions to improve performance [5], [25], [37], [66]. While this approach successfully addresses computational requirements, it fundamentally fails to reduce area and power consumption—adding instructions inevitably yields larger, not smaller, processor cores. This represents a critical missed opportunity: empirical studies of real-world workloads consistently demon-

strate that most applications utilize only a small subset of available ISA instructions [8], [47], suggesting that significant hardware resources in general-purpose processors remain unused and could be eliminated without affecting its correctness.

This insight has catalyzed the development of RISPs, designed through subtractive processes — removing ISA instructions rather than adding them — which enables systematically removing unused hardware [47]. A substantial body of work from both academia and industry has emerged to address this challenge. Bespoke microprocessors [16] demonstrated that processors can be optimized to support only a single application through automated hardware removal. PDAG [8] generalized this approach to support arbitrary ISA and microarchitecture combinations using Linear Temporal Logic (LTL) model checking to identify and eliminate unused structures. RISSP [47] applied similar principles specifically to RISC-V subsets, though constrained to a particular single-cycle microarchitecture. FSYN [28] ported PDAG to an open-source toolchain. Complementary approaches have explored approximate computing through Application Specific Approximate Behavioral Processors [54], extended symbolic execution frameworks as follow-up to Bespoke [53], applied structural subsetting to accelerators [70], and developed formal CPU profiling methods for embedded processors [60].

However, existing approaches share fundamental limitations that constrain their practical applicability and optimization potential [8], [16], [28], [47], [53], [54], [60], [70]. First, current approaches derive ISA constraints by profiling target applications, yet the compilers generating these applications remain unaware of opportunities to reduce instruction usage for additional area savings. Second, existing constraints are coarse-grained and microarchitecture-agnostic. Most works rely solely on ISA-level constraints, overlooking optimization opportunities inherent in application-specific data patterns and timing constraints. Third, most works inadequately address the trade-off between specialization and generalization. While extreme specialization yields substantial area and power benefits, minimal generalization results in high Non-Recurring Engineering (NRE) costs and severely limits future software upgrade capabilities.

To address the aforementioned challenge, we propose æSIP, a hardware-software (ASIP-ISA) co-design framework that aims at efficient and effective automatic ASIP generation. æSIP is an end-to-end framework that accepts target applications, a baseline general-purpose processor, and latency constraints as inputs, producing an optimized ASIP and verifiably equivalent rewritten programs.

æSIP uses program synthesis to discover various instruction rewriting rules. Equality saturation then applies the verified rewriting rules to generate semantically equivalent implementations of the target application. E-graphs compactly store all equivalent programs, avoiding the phase-ordering problem, where the order of applying rewriting rules can lead to different results.

The equivalent programs are then analyzed to extract microarchitecture-aware constraints, including ISA, data, and

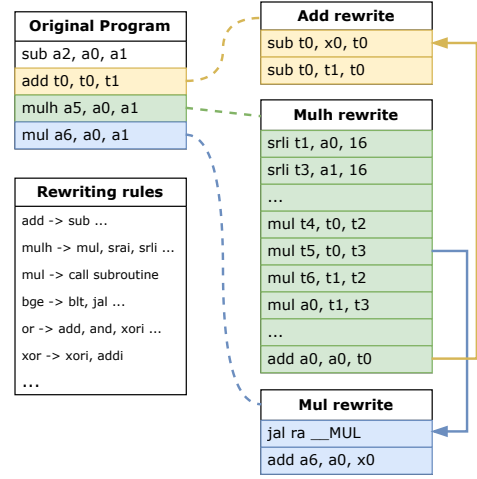


Fig. 2. Hardware-aware program rewriting for ASIP optimization. Instructions such as `mul`, `mulh`, and `add` are replaced with semantically equivalent instruction sequences, reducing instruction diversity and enabling aggressive gate-level pruning for smaller ASIP implementations. Rewrite rule details and stack allocation are omitted here due to space limit.

timing-level constraints. Using these constraints, the framework efficiently prunes the baseline general-purpose processor to generate the optimized ASIP, leveraging Observability Don't Cares (ODCs)-based optimizations and semiformal verification.

To explore the trade-off between NRE cost/upgradability and area/power of ASIPs, we develop a clustering-based algorithm to partition applications into sub-ecosystems based on their similarity. Then co-design between ASIP generation and program rewriting at the ecosystem scale enables efficient NRE amortization.

The major research contributions of this paper are as follows:

- We propose æSIP, the first e-graph-based hardware-software co-design framework that enables extreme area reduction via program rewriting.
- We use neuro-symbolic-based program synthesis to automatically derive rewrite rules for efficient equality saturation.
- We develop a divide-and-conquer rewriting framework that performs local saturation at basic-block granularity and global extraction at whole-program scope, enabling scalability to real-world benchmarks.
- We incorporate previously ignored microarchitecture-aware constraints into the ASIP generator.
- We analyze similarity across workloads and use a ASIP-ecosystem co-design to reduce NRE cost.

Upon publication, æSIP will be released as an open source design tool.

II. MOTIVATING EXAMPLE

State of the Art (SOTA) ASIP design frameworks [8], [16], [28], [47] profile target applications to gather instruction usage information, which is then translated into ISA-level constraints

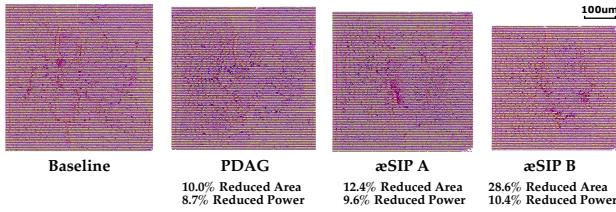


Fig. 3. Place-and-route (PnR) results for ASIPs pruned from the baseline Ibex RISC-V core in 130nm technology. PDAG [8] represents the SOTA RISP design methodology, which profiles target applications and prunes the baseline processor based on ISA constraints. æSIP A and B represent two variants with different program rewriting strategies. More aggressive rewriting achieves greater area/power reduction at the expense of increased latency.

for pruning unused gates to obtain smaller ASIP designs. However, existing compilers focus on performance optimization rather than reducing instruction usage to reduce chip area. We observe that strategic program rewriting can significantly reduce instruction diversity, thereby enabling more aggressive hardware pruning.

Figure 2 illustrates this opportunity through a concrete example in RV32IM instruction set. The `mulh` instruction, which computes the upper bits of a multiplication, can be eliminated by rewriting it as a sequence using `mul` (computing lower bits) combined with appropriate shifts and accumulations. Similar transformations apply to `mulhu` and `mulhsu` instructions. This rewriting enables the safe removal of multiplier logic dedicated to upper-bit computation. Furthermore, `mul` itself can be decomposed into a subroutine consisting solely of additions and shifts, potentially eliminating the entire multiplier unit. The resulting ASIPs, due to different ways of program rewriting, are shown in Figure 3.

While this approach demonstrates clear area benefits, it raises several fundamental challenges:

- **Correctness verification:** We must systematically derive and verify semantic-preserving rewriting rules.
- **Phase ordering problem:** The application order of rewrite rules critically affects optimization outcomes. In our example, `mulh` must be rewritten before `mul` to achieve complete elimination of multiplication hardware.
- **Latency/Area trade-off:** Decomposing complex instructions into primitive operations reduces area at the cost of program latency. Different applications impose varying latency constraints that must be respected.
- **Scalability:** The approach must scale to large, real-world workloads while maintaining tractability. Program rewriting may lead to combinatorial explosion, as each rewriting step can potentially introduce additional rewriting opportunities, creating a cascading effect that becomes computationally intractable.

To address these challenges comprehensively, we propose æSIP, an ASIP-ISA co-design framework that combines program synthesis, equality saturation, and external don’t-care-based hardware pruning. To the best of our knowledge, this represents the first systematic approach to minimizing ASIP area through hardware-aware program transformation.

III. BACKGROUND

A. Program Synthesis

Program synthesis provides a principled way to search the space of semantics-preserving program transformations. Program synthesis is commonly used in code super-optimizers [1], [45]. Modern synthesis techniques fall broadly into three categories:

1) *Symbolic synthesis:* Symbolic approaches encode the search problem as logical constraints, typically using SAT/SMT solvers. Existing works like Sketch and Rosette [58], [63] demonstrate how symbolic reasoning enables automated derivation of correct-by-construction transformations. These synthesizers treat the program synthesis problem directly as a search for a constructive proof of a second order existential logic theorem — they must prove (by construction) that there exists a program which, for all possible inputs, satisfies the intended semantics. This gives it a complexity of Σ_3^0 -complete in the arithmetic hierarchy [33].

2) *Stochastic and heuristic synthesis:* Due to the significant computational overhead of symbolic synthesis, *stochastic* superoptimizers (e.g., STOKE [52]) explore the program space using Monte Carlo search, simulated annealing, or genetic mutations. They trade optimality for scalability and can discover non-obvious rewrites that are difficult for symbolic methods to reach.

3) *Enumerative synthesis:* Enumerative techniques systematically enumerate candidate expressions under a bounded grammar. Although exponential in the worst case, recent pruning and version-space techniques make enumerative synthesis practical for domain-specific instruction-level rewrites [3].

B. E-Graphs and Equality Saturation

E-graphs provide a compact and canonical representation for large sets of equivalent expressions. Originally developed for automated theorem provers (ATPs) [21], the data structure has recently gained renewed attention with the advent of state-of-the-art open-source equality-saturation frameworks such as egg [69] and egglog [73]. These systems demonstrate that e-graphs are highly effective across a wide range of compiler and synthesis tasks, including code optimization [14], [72], technology mapping [12], [13], [71], circuit datapath synthesis [18], [19], arithmetic reasoning [17], [35], [42], [65], and more.

An e-graph consists of *e-nodes* (individual operators) and *e-classes* (sets of equivalent nodes). The overall workflow comprises two phases: saturation and extraction. Given an initial program representation and a rewrite rule set, equality saturation repeatedly applies all applicable rewrite rules to the e-graph until a fixed point is reached. In the subsequent extraction phase, a user-defined cost model is used to select a combination of e-nodes, yielding an optimized program that is guaranteed to be semantically equivalent to the original.

Leveraging the non-destructive nature of e-graphs, AESIP performs assembly-level program rewriting without phase-ordering issues. It systematically explores the full space of

TABLE I
REPRESENTATIVE REWRITING RULES IN $\mathcal{A}\mathcal{E}\mathcal{S}\mathcal{I}\mathcal{P}$

Class	Original	Rewritten
I-type	xori	xor rs, (addi x0, imm)
	slli	sll rs, (addi x0, imm)
	slti(u)	slt(u) rs, (addi x0, imm)
Arithmetic	or	xor (xor rs1, rs2), (and rs1, rs2)
	add	sub rs2, (sub x0, rs1)
	and	sub rs1, (sub (or rs1, rs2), rs2))
	xor	sub (or rs1, rs2), (and rs1, rs2)
Branch	blt(u)	seq (bge(u) rs1, rs2, .+8), (jal target)
	bne	seq (beq rs1, rs2, .+8), (jal target)
	jal	beq a a target
Mult/Div	mul	callmul sequence using <add, addi, andi, beq, srli, slli, bne, jalr>
	mulh(s) (u)	mul sequence using <lui, addi, and, srli, mul, add, (srai), (sub)>
	div(u)	calldiv(u) sequence using <blt, beq, bne, bgeu, bge, bltu, blt, addi, slli, sub, or, auipc, jalr>
Ld/St	lh	inline sequence using lw, srli, slli, srai
	lhu	inline sequence using lw, srli, andi

program variants that induce different ISA subsets, thereby directly guiding ISA subset generation and downstream ASIP design.

C. External Don't Care

Don't cares (DCs) [11] are a well-established technique in logic synthesis that allows minimizing logic complexity by exploiting conditions where certain values or behaviors are irrelevant. Don't cares are typically categorized into Internal Don't Cares [11], [51], which arise from the netlist structure itself, and External Don't Care (EDC) [34], [51].

EDC represent constraints from the user specifications that define input patterns that will never appear in practice. They present a unique optimization challenge because they transform the problem from optimizing a completely specified Boolean function to optimizing a Boolean relation, enabling potentially substantial gate reductions.

IV. METHODOLOGY

In this section, we describe the implementation details of $\mathcal{A}\mathcal{E}\mathcal{S}\mathcal{I}\mathcal{P}$, following the order of its workflow as shown in Figure 4.

A. Program Synthesis Based Rewrite Rules Discovery

Manually creating rewriting rules for each instruction in RV32IM is both tedious and error-prone [36]. To systematically discover rewrite rules that are both correct and effective for gate removal, we developed a dedicated tool for synthesizing rewrite rules for individual instructions using an existing code superoptimizer, Greenthumb [45]. We extended it to support RV32IM. The search algorithms, including symbolic, stochastic, and enumerative approaches, are automatically inherited from the original framework with minimal programming effort.

In RV32IM program state is modeled as an array of registers and memory, $\text{State} = (\mathcal{R}, \mathcal{M})$ where $\mathcal{R} : \{0, \dots, 31\} \rightarrow \mathbb{B}^{32}$ and $\mathcal{M} : \mathbb{B}^{32} \rightarrow \mathbb{B}^8$

Then each instruction that transforms between program states: $\llbracket \text{inst} \rrbracket : \text{State} \rightarrow \text{State}$.

Instruction sequence therefore can be constructed as:

$$\llbracket S \rrbracket = \llbracket \text{inst}_n \rrbracket \circ \dots \circ \llbracket \text{inst}_1 \rrbracket$$

Finding rewrite rules thus involves identifying instruction sequences, S_{synth} that produce the same state transformations as the original instruction, S_{spec} .

To verify equivalence, following relation needs to be proved:

$$S_{\text{spec}} \equiv_L S_{\text{synth}} \iff \forall \sigma, \pi_L(\llbracket S_{\text{spec}} \rrbracket(\sigma)) = \pi_L(\llbracket S_{\text{synth}} \rrbracket(\sigma))$$

By precisely defining each instruction's effect on the program state, we can translate instruction sequences into SMT formulas and prove equivalence using SMT solvers such as Z3 [21].

Our approach differs from existing tools in two key aspects. First, we aim to rewrite single instructions into potentially longer sequences using other instructions. Second, rather than optimizing for performance, our goal is to find equivalent sequences that reduce overall distinct instruction usage. To achieve this, we constrain the available instruction set to exclude the original instruction being rewritten. More importantly, we prioritize instruction subsets that utilize different hardware components, ensuring that the hardware required by the original instruction can be safely removed to reduce total chip area.

Given the complexity of rewriting individual RISC-V instructions, we further incorporate a lightweight neurosymbolic [67] refinement loop that uses an LLM to propose candidate rewrite patterns. Recent work has shown that LLMs can capture structural algebraic transformations that are difficult to enumerate symbolically [33], [35], [41], [43], [64]. We warm-start the LLM with a small set of expert-verified rewrite examples, enabling it to generalize common patterns and propose semantically plausible replacements. Each candidate is then checked using SMT, and counterexamples generated by the solver are fed back to the LLM, forming a counterexample-guided neurosymbolic synthesis loop similar to recent approaches in hybrid theorem-proving and optimization. This synergy helps the search discover longer or less intuitive rewrites while maintaining full formal correctness.

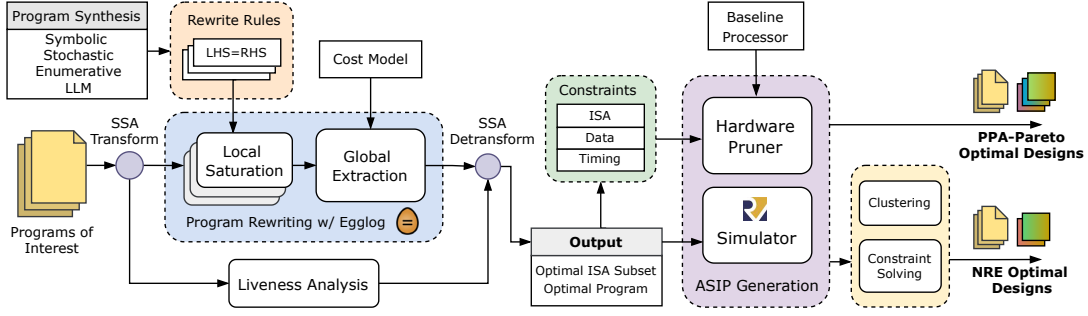


Fig. 4. Overview of the æSIP workflow. Program synthesis is invoked once to derive verified rewrite rules. For given programs of interest, equality saturation generates multiple semantically equivalent rewritten programs. Profiling then automatically infers microarchitecture-level constraints, which are subsequently incorporated into ASIP generation. Optionally, constraint-aware optimization can be applied to enable NRE exploration.

Through the cooperative effort of all four search algorithms, we successfully obtained the rewrite rules. Example rewrite rules are shown in Table I.

B. E-Graph Based Hardware-Aware Program Rewriting

1) *Preprocessing*: We select well-studied embedded workloads from standard benchmark suites [29], [44]. Several applications invoke small library routines (e.g., string manipulation or math helpers). To ensure that rewriting and instruction-subset extraction operate on a closed program, we manually inline these library functions into the application’s source file. We then compile all benchmarks using the RV32IM toolchain. Because the compiler emits pseudo-instructions (e.g., `li`, `mv`) that are not directly implemented in hardware, we analyze the generated assembly and expand all pseudo-instructions into their canonical sequences of real RISC-V instructions. This produces a self-contained assembly program whose semantics exactly match the final ELF binary.

This pre-expanded assembly is a complete and self-contained program, with all pseudo-instructions expanded and no external symbols. It forms the direct input to our rewriting engine. Consequently, all extracted variants remain valid standalone `.s` files that assemble into runnable binaries, which disassembly-based rewriting cannot produce.

2) *Assembly-Level Local Saturation*: Traditional compilers suffer from the well-known phase-ordering problem: the effectiveness of an optimization depends critically on the order in which transformations are applied. Equality saturation addresses this issue by applying all rewrite rules simultaneously within an e-graph, ensuring that all semantically equivalent program variants are represented.

However, applying full-program equality saturation directly to real applications quickly becomes infeasible. Rewrite rules interact non-locally and recursively expand the search space, often resulting in combinatorial blow-up. Prior work either performs coarse-grained saturation over high-level IRs [14], [72] or restricts saturation to small local regions applied sequentially [50], rather than allowing every instruction across the entire program to participate in rewriting concurrently.

In our setting, the objective is not traditional performance optimization but to explore instruction-level rewrites that elim-

inate, replace, or simplify ISA operations, thereby revealing additional opportunities for microarchitectural hardware trimming. This shifts the challenge: we do not need sophisticated control-flow-aware optimizations, but we must explore a vast equivalence space efficiently. To make this tractable, we develop a divide-and-conquer rewriting framework that accelerates both saturation and extraction. Fig. 5 shows an overview of this workflow.

We first analyze the program’s control flow and partition the entire application into a set of basic blocks. Our rewriting engine then performs structured local saturation within each basic block rather than saturating the full program monolithically. For every basic block, we create a pseudo-root e-class that serves as the anchor for all expressions derived within that block. These locally saturated subgraphs are subsequently merged into a single global e-graph by connecting all pseudo-roots to a global root e-class. This construction captures the full program’s equivalence space while avoiding the blow-up associated with whole-program saturation, and enables extraction to consider inter-block alternatives within one unified global e-graph.

In addition, any orphan e-classes—those that do not participate in a block’s primary expression tree but nonetheless correspond to values used across basic blocks—are explicitly linked back to the block’s pseudo-root. This prevents the extraction phase from incorrectly treating cross-block dependencies as dead code and ensures that instructions whose results are consumed outside the local block are preserved.

3) *ILP based Global Extraction*: Local saturation builds rich equivalence spaces within each basic block, but selecting a concrete program requires globally consistent decisions. Traditional greedy extraction, which chooses the locally cheapest e-node in each e-class, is insufficient because the objectives we optimize depend on global instruction choices.

To address this, we cast extraction as a lightweight global optimization problem and solve it using ILP [2]. We model the e-graph as a set of e-classes $\mathcal{E}$, where $\mathcal{N}(e)$ denotes the set of e-nodes in e-class $e \in \mathcal{E}$ and $\mathcal{O}$ denotes the set of instruction types. For each e-node $n \in \mathcal{N}(e)$ we introduce a binary decision variable $x_{e,n}$, while each e-class e is associated with a binary activation variable a_e indicating whether it is

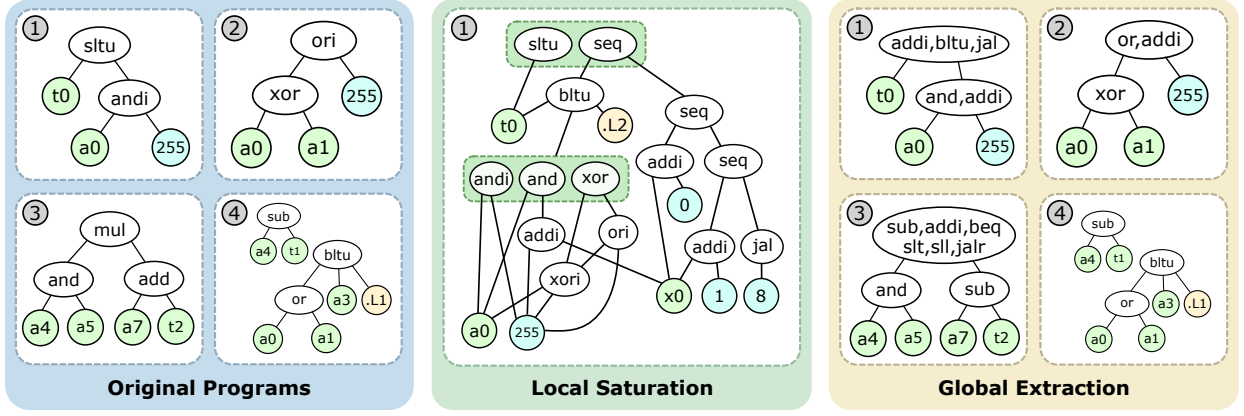


Fig. 5. Overview of the divide-and-conquer rewriting workflow. The program is first decomposed into basic blocks (left). Each block then undergoes local equality saturation to enumerate its semantically equivalent variants (middle). Finally, a global ILP-based extractor jointly selects one variant per block, minimizing instruction-type diversity under latency constraints (right) and producing a globally optimized ISA subset. For clarity, the middle panel shows only Block 1 after a single round of saturation, and the visualization simplifies several egg semantics (e.g., omitting e-class boxes for single-node expressions). Likewise, the extraction panel illustrates only the instruction types chosen for each block rather than the full reconstructed expression graphs.

selected in the extracted program. We also introduce a binary variable y_o for each instruction type $o \in \mathcal{O}$, indicating whether any e-node using type o is selected.

Let $c_{e,n}$ denote the local cost (e.g., latency) of e-node n , and let w_o denote a hardware-aware weight that captures the area/power cost of instruction type o (e.g., multipliers and dividers are assigned much larger w_o than simple ALU operations).

We define the ILP objective as:

$$\min_{x,a,y} \lambda \sum_{o \in \mathcal{O}} w_o y_o + (1 - \lambda) \sum_{e \in \mathcal{E}} \sum_{n \in \mathcal{N}(e)} c_{e,n} x_{e,n}, \quad (1)$$

where the first term minimizes the set of distinct instruction types used and the second term (scaled by a user-chosen factor λ) minimizes their total usage count. By adjusting the relative scale between $\{w_o\}$ and λ , we can bias extraction toward variants that minimize hardware area, reduce program latency, or balance the two. In practice, we sweep a broad range of weight configurations to generate multiple program variants, enabling exploration of the area–latency tradeoff for the ASIP tailored to this application.

The constraints are summarized as follows. Let $\mathcal{R} \subseteq \mathcal{E}$ denote the set of pseudo-root and orphan e-classes that must be extracted, let $\text{child}(n)$ be the set of child e-classes of e-node n , and let $\text{op}(n) \in \mathcal{O}$ be the instruction type of n . First, each activated e-class must select exactly one e-node:

$$\forall e \in \mathcal{E} : \sum_{n \in \mathcal{N}(e)} x_{e,n} = a_e. \quad (2)$$

All pseudo-root and orphan e-classes are forced to be active:

$$\forall r \in \mathcal{R} : a_r = 1. \quad (3)$$

If an e-node is selected, all of its child e-classes must be active:

$$\forall e \in \mathcal{E}, \forall n \in \mathcal{N}(e), \forall e' \in \text{child}(n) : x_{e,n} \leq a_{e'}. \quad (4)$$

If an e-node is selected, the corresponding instruction type must also be enabled:

$$\forall e \in \mathcal{E}, \forall n \in \mathcal{N}(e) : x_{e,n} \leq y_{\text{op}(n)}. \quad (5)$$

Finally, we forbid invalid self-cycles by assigning each e-class an integer “level” ℓ_e and enforcing a strict topological order whenever a parent–child dependency is selected:

$$\forall e \in \mathcal{E}, \forall n \in \mathcal{N}(e), \forall e' \in \text{child}(n), \\ x_{e,n} = 1 \Rightarrow \ell_{e'} \geq \ell_e + 1 \quad (6)$$

which we encode using a standard Big-M linearization in the actual ILP.

Because pseudo-root and orphan e-classes are explicitly forced active by (3), their corresponding basic blocks must be extracted. The remaining constraints (2)–(6) follow the standard formulation used in existing e-graph extractors, while our additional operator-type variables y_o and weights w_o make the optimization hardware-aware. Under these constraints, the ILP solver naturally decomposes the optimization across basic blocks: each block has its own root e-class and local e-nodes, but all blocks share the same global operator-type variables y_o . This implicit block-level parallelism substantially accelerates global extraction while preserving full semantic correctness, reducing overall runtime from hours under monolithic extraction to seconds in our parallel block-wise workflow.

4) *Postprocessing*: Given the selected e-classes and e-nodes from the global extraction phase, we reconstruct the program by rebuilding the corresponding expression graph and performing a postorder traversal to emit concrete assembly instructions. Our framework automatically tracks operand registers and assigns destination registers by inheriting the original instruction’s `rd` whenever the rewritten instruction belongs to the same e-class.

When a single instruction is rewritten into a longer sequence, additional temporary registers may be required. To allocate these registers safely, we perform a control-flow-aware

analysis of register definitions and uses to compute register liveness within each basic block. Temporaries are first drawn from the set of free registers; when insufficient, we borrow registers by spilling to the stack.

Naively assigning a distinct temporary register for every intermediate value is the simplest approach, but it is often impractical. For example, replacing a `div` instruction with a multi-instruction RV32I sequence can introduce so many live temporaries that the allocator runs out of the entire architectural register file. To avoid such register pressure, we implement a lightweight register allocation algorithm within each basic block. The allocator performs linear-scan-style [46] liveness analysis over the rewritten instruction sequence, reusing temporaries whenever their live ranges do not overlap. When multiple intermediates are alive simultaneously, the allocator prioritizes allocating architectural registers with the shortest remaining live ranges; only when these are exhausted does it introduce stack-backed spill slots. This localized allocation strategy ensures that even multi-instruction rewrites remain executable without exceeding the architectural register budget.

C. Don't Care Based μ Arch-Aware Agile ASIP Generator

Beyond ISA-level constraints, we leverage microarchitecture-aware constraints to achieve further reductions in chip area. These constraints capture implementation-specific behaviors that are guaranteed by the processor's design and fine-grained application characteristics. We categorize these constraints into data-level and timing-level constraints, both of which serve as EDC [38] that enable aggressive gate-level pruning.

1) *Data-Level Constraints*: Data-level constraints arise from the observation that certain instructions operate on restricted data ranges despite having wider architectural support. These constraints manifest in several forms:

Operand Width Constraints: Many instructions in practice operate on data types narrower than the processor's word width. For instance, string processing routines frequently manipulate only `char` (8-bit) or `short` (16-bit) values within 32-bit registers. When certain instructions never utilize the full register width in specific applications, the corresponding arithmetic units can be simplified. For example, if analysis shows that a particular multiply instruction only processes 16-bit operands, the upper multiplication logic becomes redundant and can be safely removed.

Immediate Value Constraints: Instructions with immediate operands often use only a small discrete subset of possible values. Shift instructions exemplify this pattern—while the ISA may support wide shift amounts, profiling reveals that only specific values appear in specific applications. This observation enables specialized, area-efficient implementations. Furthermore, program rewriting can intentionally strengthen these constraints. For instance, if `srai` (shift right arithmetic immediate) operations use only values 2, and 4, we can decompose them into sequences of `srai` by 2, with a minor latency penalty for area savings if `srai` by 4 happens rarely.

2) *Timing-Level Constraints*: Timing-level constraints capture the temporal behavior of memory subsystems and other microarchitectural components. These constraints reflect design-specific guarantees about latencies and access patterns. For example, we might assert that data cache hits complete within one cycle while misses resolve within five cycles. Similarly, instruction cache behavior can be bounded. These constraints enable simplification of memory interface state machines, removing states and transitions that handle scenarios outside our specified bounds.

It is crucial that users validate these timing constraints against their specific application workloads and baseline processor microarchitecture. Invalid constraints could lead to incorrect behavior in scenarios not covered by the validation suite.

3) *Constraint Specification and Application*: We developed an automated tool that analyzes each program variant generated by our rewriting system and derives microarchitectural constraints. These constraints are systematically converted into SystemVerilog assertions that formally specify the EDC. Representative assertions include:

ISA constraint
<pre># Only Allow Some opcodes. assume ((i_rdata[1:0] != 2'b11) ((((i_rdata[31:0] & 32'hfe00707f) == OP_ADD)) (((i_rdata[31:0] & 32'h0000707f) == OP_SUB)) (((i_rdata[31:0] & 32'hfe00707f) == OP_MUL))))</pre>
Data Constraint
<pre># Shift right amount must be 1 or 2. assume ((opcode == SRAI) > (shamt inside {1, 2}));</pre>
Timing Constraint
<pre># Data cache misses are resolved within five cycles. assume (dcache_miss > ##[1:5] dcache_response);</pre>

4) *Semiformal Verification for Gate Pruning*: The critical challenge in hardware pruning is determining which gates can be safely removed under the specified EDCs without affecting circuit correctness. Exhaustive verification through complete state-space exploration is computationally intractable for industrial-scale designs. Particularly, LTL Model Checking is PSPACE-Complete for fixed model, EXPTIME-Complete with respect to the number of flops in model, uses state-space methods, make it practically impossible to be used for large-scale design space exploration [8], [30].

We therefore employ a semiformal verification approach based on k -step induction [39] that provides soundness guarantees while achieving practical verification times. We use the existing implementation `scorr` in the `abc` [27], [39] framework. The verification proceeds in two phases:

Base Case (BMC): Unroll the circuit for k time frames starting from the initial state and use Bounded Model Checking (BMC) to exhaustively verify that the redundancy property in the first k frame starting from the initial state.

Inductive Step: Prove that if the redundancy property holds for *any* sequence of k consecutive states, it must also hold for

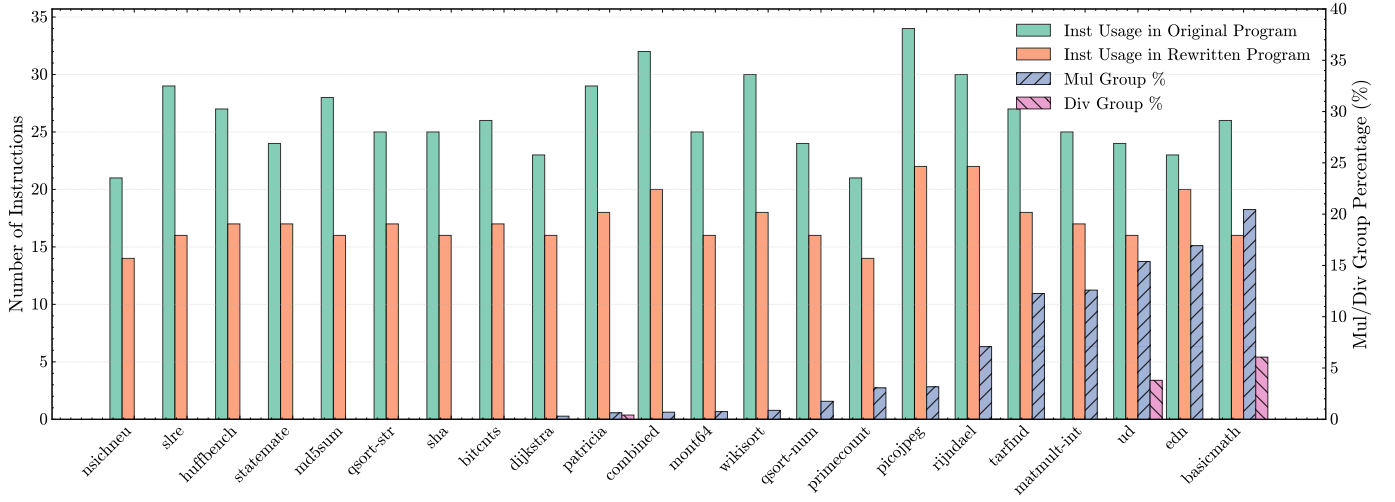


Fig. 6. Comparison between original program and æSIP rewritten programs in embedded system applications in RV32IM. Up to 44.8% instructions can be safely removed without affecting functional correctness of target program. Besides, instructions execution times related to multiplication and division are explicitly shown due to their large impact on chip area.

the $(k + 1)$ -th state.

With semiformal verification, we sacrifice completeness for scalability. There may exist gates that are truly redundant but cannot be proven so via k -step induction for a given bound k . This occurs when the redundancy only becomes apparent after observing more than k consecutive states. After experiments, we find that modest values of k (typically 2-5) suffice for identifying the vast majority of redundant gates while maintaining reasonable verification times.

D. End-to-End Verification

As a hardware–software co-design framework, æSIP is designed to be practical and genuinely end-to-end: beyond generating aggressively pruned ASIPs, the resulting processors must remain fully programmable. This stands in contrast to prior work on ISA customization or instruction fusion, where validation is often limited to Spike-level trace matching and software compatibility is frequently ensured only by ad-hoc compiler patches or specialized backends. In æSIP, every extracted ISA subset is guaranteed to remain a drop-in replacement for the baseline ISA, and all rewritten programs compile and execute using the unmodified RV32IM toolchain.

To validate this property, we perform full-system correctness testing on eight applications selected from the automotive, security, and network domains of MiBench [29]. For each application, we verify functional equivalence across all stages of our pipeline: the original program, the preprocessed assembly, and the final e-graph–rewritten output. We execute the rewritten binaries on Spike together with `pk`, capturing `printf` outputs to confirm identical program behavior. All rewritten binaries produce results that match the original executions exactly, demonstrating that our ISA-subset–driven rewriting preserves complete software correctness.

E. Hardware-Software-Ecosystem Codesign for NRE Reduction

A well-known challenge in ASIC and ASIP development is the substantial Non-Recurring Engineering (NRE) cost associated with tapeout, verification, and tool-flow customization [20], [24], [57], [62]. While extensive prior work [5], [8], [31], [47], [48] has focused on minimizing design effort through automated microarchitectural pruning or ISA customization, these approaches still fundamentally target a single application, thereby limiting amortization of the NRE cost across different workloads. In many embedded domains—such as automotive control, IoT sensing, or edge security, developers must support a family of related workloads rather than a single static binary. This motivates the need for an architectural solution that balances specialization, which yields area and power benefits, and generalization, which ensures longevity and cost-effectiveness over an evolving software ecosystem, as also suggested by prior works [32].

æSIP introduces a hardware–software–ecosystem co-design strategy that explicitly explores this trade-off. Given a suite of target workloads, we profile their instruction usage patterns and construct high-level feature vectors that capture instruction types, dynamic frequencies, and operand characteristics. These embeddings are clustered using a K-means–style algorithm to identify subsets of workloads that naturally share microarchitectural affinities. For each cluster, æSIP performs program rewriting collectively, discovering a unified ISA subset that can support all programs in the cluster while still enabling aggressive hardware pruning. In this way, a single ASIP instance can serve an entire sub-ecosystem of applications, significantly amortizing NRE cost without sacrificing the benefits of specialization.

This ecosystem-level co-design also provides robustness to software evolution. When new applications arrive (e.g., program B that is structurally different from program A) the

TABLE II
PERFORMANCE COMPARISON OF ORIGINAL AND OPTIMIZED ASIP DESIGNS

Application	PDAG [8]				æSIP							
	Area (μm^2)	Power (mW)	Freq (MHz)	Latency (s)	Area (μm^2)	Power (mW)	Freq (MHz)	Latency (s)	Area (μm^2)	Power (mW)	Freq (MHz)	Latency (s)
basicmath	126625	3.12	20.7	0.48	89059	-29.7%	2.86	-8.3%	26.1	+25.7%	111.88	+23087%
bitcnts	111680	2.95	23.0	1.52	86928	-22.2%	2.83	-4.1%	27.3	+18.6%	1.28	-15.6%
combined	118561	2.95	22.9	0.09	89680	-24.4%	2.89	-2.0%	26.0	+13.3%	0.91	+863%
dijkstra	117569	2.94	24.6	1.00	88874	-24.4%	2.86	-2.7%	27.2	+10.4%	5.06	+408%
edn	109800	2.95	23.2	0.15	89431	-18.6%	2.88	-2.4%	26.7	+15.2%	7.48	+4956%
huffbench	89367	2.84	27.1	0.08	86843	-2.8%	2.83	-0.4%	27.9	+3.0%	0.08	-2.9%
nsichneu	89030	2.86	27.7	0.07	86801	-2.5%	2.83	-1.0%	28.2	+1.8%	0.07	-1.7%
slre	89336	2.87	27.6	0.09	87079	-2.5%	2.85	-0.7%	27.8	+0.8%	0.09	-0.8%
statemate	89716	2.86	26.7	0.05	87531	-2.4%	2.84	-0.7%	26.7	-0.1%	0.05	+0.1%
ud	122885	3.12	20.6	0.04	89208	-27.4%	2.86	-8.3%	27.2	+31.6%	2.07	+5116%
wikisort	125986	3.13	21.8	0.03	89312	-29.1%	2.89	-7.7%	27.1	+24.3%	0.27	+699%
matmult-int	118096	2.94	23.9	0.13	89277	-24.4%	2.85	-3.1%	26.9	+12.7%	4.93	+3825%
md5	89337	2.86	26.4	0.07	89159	-0.2%	2.88	+0.7%	28.0	+6.0%	0.07	-4.9%
mont64	110885	2.93	22.4	0.20	89159	-19.6%	2.88	-1.7%	27.0	+20.7%	1.77	+780%
patricia	123360	3.12	20.9	0.24	89406	-27.5%	2.86	-8.3%	26.4	+26.1%	1.01	+321%
picojpeg	111243	2.96	24.1	0.16	90024	-19.1%	2.86	-3.4%	27.5	+14.3%	1.71	+959%
primecount	111551	2.93	23.7	0.18	88840	-20.4%	2.88	-1.7%	27.4	+16.0%	1.81	+895%
qsort-num	117757	2.94	23.2	0.74	88874	-24.5%	2.86	-2.7%	27.2	+17.4%	18.13	+2358%
qsort-str	89362	2.86	27.8	1.07	86787	-2.9%	2.86	0.0%	28.1	+1.3%	1.06	-1.2%
rijndael	117843	3.11	20.8	32.96	107783	-8.5%	2.93	-5.8%	22.8	+9.4%	30.13	-8.6%
sha	89366	2.86	27.4	0.48	86966	-2.7%	2.84	-0.7%	28.2	+2.9%	0.46	-2.8%
tarfind	111238	2.95	21.9	0.04	89398	-19.6%	2.86	-3.1%	26.9	+22.8%	4.97	+11957%

æSIP rewriting engine can attempt to rewrite B into the ISA subset designed for A’s cluster. This flexible accommodation of new or atypical workloads allows æSIP to maintain high specialization efficiency while avoiding unnecessary proliferation of ASIP variants.

V. EVALUATION

In this work, we use `ibex` as the target general-purpose processor for optimization. We evaluate our approach on widely-studied embedded system workloads from MiBench and EmBench [29], [44]. We employ `egglog` as our equality saturation engine [69], [73]. Our program synthesis extends the Greenthumb superoptimizer [45] and is thus built on Rosette and Racket [23], [63]. Our compilation and profiling framework leverages the open-source `riscv-gnu-toolchain` [49]. We place and route our designs with SkyWater 130nm library [56] using Synopsys fusion compiler [59] to obtain iso-frequency power results at 25MHz. Frequency analysis is done via OpenSTA [15].

A. Profiling of Embedded System Workload

In this section, we profile commonly used embedded system applications to analyze instruction usage patterns in RV32IM. Specifically, we measure the execution frequency of instructions in the Mul group (`mul/mulh/mulhu/mulhsu`) and Div group (`div/divu/rem/remu`), as their corresponding functional units consume substantial chip area. Execution counts are collected using `spike` and `pk`. We then evaluate the extent of instruction elimination achievable through our e-graph-based program rewriting framework.

As illustrated in Figure 6, each program utilizes only a subset of all 45 instructions supported by RV32IM. SOTA ASIP design frameworks such as PDAG [8] and RISSP [47] leverage this observation to prune unused hardware components. Our program rewriting framework achieves up to

44.8% instruction reduction, while the backend ODC-based hardware pruner enables additional area optimization. Notably, we observe substantial variation in Mul/Div usage across different programs. For instance, Mul/Div instructions only make up 0.0002% of the runtime for `bitcnts` and 0.3055% the runtime for `dijkstra`. Given the considerable area overhead of multiplier and divider units, rewriting these instructions using simpler primitives presents an opportunity for significant area reduction with acceptable latency trade-offs.

B. ASIP for Embedded System Application

In this section, we present the ASIPs optimized using æSIP and compare against SOTA ASIP design frameworks. We select PDAG [8] as the representative ASIP design methodology, which prunes unused hardware from the baseline processor using ISA-level constraints derived from software profiling. For each embedded system application, we apply our hardware-aware program rewriting engine with a careful tuned cost function reflecting the impact of each program rewrite on hardware cost. ISA and data-level constraints are derived from profiling the rewritten program, while timing-level constraints remain as described in Section IV. Among all generated variants, we report the smallest-area design.

The general-purpose `ibex` core supporting the full RV32IM ISA yields $126\,176\mu\text{m}^2$ at 22 MHz, which serves as comparison baseline for all ASIP variants. As shown in Table II, SOTA ASIP design frameworks achieve substantial area reduction when programs utilize only small instruction subsets, particularly those excluding complex instructions like `mul` or `mulh`. For instance, `nsichneu` achieves 29% area reduction as it uses only 21 instructions without complex operations. However, programs like `bitcnts` and `dijkstra` achieve only 11% and 7% area reduction respectively, as they require complex instructions like `mul`, despite their low usage.

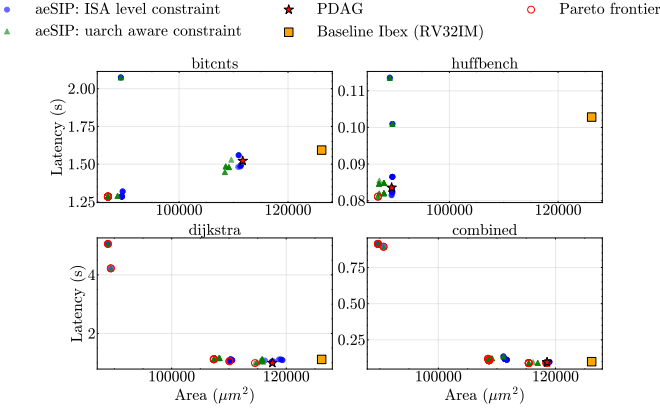


Fig. 7. Selected Pareto curves showing latency-area trade-offs. Different curve shapes reflect varying instruction usage characteristics across applications. Removing support for complex instructions can be Pareto-dominant in both latency and area when such instructions are rarely used.

æSIP, on the other hand, achieves up to 29.7% and on average 16.2% additional area reduction compared to PDAG. This improvement stems from two key factors. First, program rewriting significantly reduces instruction usage, enabling more aggressive data-level constraints and area reduction. By configuring the cost model—adjusting weights for specific instructions based on hardware costs—we guide rewrites toward effective area reduction. For example, rewriting `mulh/mulhu/mulhsu` using convolution or Karatsuba algorithms [68] eliminates hardware gates for upper-bit multiplication. Second, combining data-level and timing-level constraints unlocks optimization opportunities even when rewrites are inapplicable. Integrating microarchitecture-aware constraints enables fine-grained optimizations including memory system state machine and barrel shifter simplification.

Program rewriting inevitably increases cycle count despite area reduction, particularly when generating complex instruction sequences with high runtime execution. However, certain workloads achieve both smaller area and shorter latency. For instance, `bitcnts` achieves 22.2% area reduction and 15.6% latency reduction simultaneously. This occurs because rewriting reduces the processor’s critical path, enabling higher frequency while the rarely-used rewritten instruction (`mul`) causes minimal cycle count impact, compensating through frequency gains. We defer comprehensive frequency and power trade-off exploration to future work.

C. Area-Latency Trade-off Analysis

In this section, we present the latency-area trade-off exploration supported by æSIP. While tight area/power constraints remain bottlenecks for many embedded system applications, different application scenarios impose varying latency constraints [7], [9], [47]. æSIP achieves aggressive hardware pruning through program rewriting, which generally increases cycle count. For example, rewriting `mulh` requires up to 20 instructions including but not limited to `mul`, `shift`, and `add`. Rewriting `mul` using `shift` and `add` further

expands execution to up to 300 cycles. When these complex instructions have significant usage, the rewritten program’s latency may become unacceptable for certain applications.

To address these concerns, æSIP explores the latency-area trade-off by tuning λ in the cost model: $\lambda \times Area + (1 - \lambda) \times Latency$, which guides program rewriting and generates multiple ASIP variants. Each variant comprises its rewritten program, area, frequency, and latency. The resulting Pareto frontier based on area and latency enables architects to select variants tailored to their requirements.

Figure 7 presents four representative Pareto curves. Workloads like `bitcnts` and `huffbench` achieve both smaller area and shorter latency—`mul` appears in only 0.0002% and 0% of total execution time, respectively. Thus, rewriting `mul` introduces minimal cycle penalty while the reduced critical path increases frequency, yielding lower latency. For `dijkstra` and `combined`, completely removing all complex instructions achieves 24.4% area reduction but incurs 4x and 8x latency increases, potentially unacceptable for some applications. However, intermediate variants exist by selectively rewriting `mulh`, immediate-value instructions, large-amount shifts and so forth, achieving up to 10% area reduction with minimal latency penalty due to frequency increases.

D. Ecosystem-Level ASIP Sharing and NRE Reduction

While the previous sections evaluate per-application ASIPs, practical deployment requires amortizing design cost across multiple workloads. æSIP’s hardware–software–ecosystem co-design explicitly targets this requirement by assigning similar applications to a shared ASIP variant and jointly optimizing rewriting and ISA-subset extraction at the cluster level, rather than for each program in isolation.

To motivate this design, we parse the basic blocks of each RV32IM program, build instruction histograms, and derive feature vectors that encode (i) which instruction types appear and (ii) their normalized dynamic frequencies. These vectors are standardized, clustered with K-means, and then projected to two principal components for visualization. Fig. 8 shows that despite their diversity, embedded workloads share recurring structural patterns that naturally give rise to several meaningful clusters, enabling practical ASIP sharing across programs.

To concretely evaluate ecosystem-level sharing, we use the full set of program variants generated earlier by the rewriting pipeline. Each program may select exactly one variant, and our goal is to determine how many ASIP designs are required to cover the union of their ISA subset requirements. We formulate this as a constrained optimization problem: given a target number of ASIP instances, we assign programs to ASIPs and solve a global extraction problem that enforces a unified ISA subset within each ASIP group, i.e., all programs assigned to the same ASIP must share the same ISA subset.

We sweep the allowed number of ASIP variants (`num-chip` = 0...9) and run the ILP-based global extractor under each configuration. Here, `num-chip` = k means that exactly k distinct ISA subsets must collectively support all workloads, requiring the optimization to jointly choose

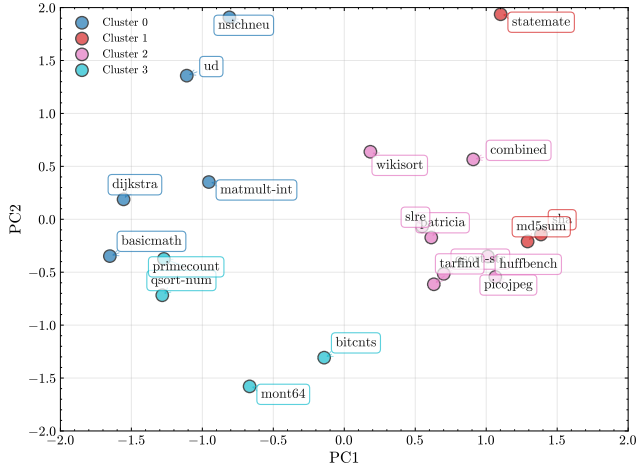


Fig. 8. PCA projection of embedded workloads based on instruction-usage feature vectors (instruction presence and normalized frequency). Programs naturally form a handful of clusters, indicating that many benchmarks might share a common rewritten ISA subset and ASIP implementation.

program variants whose rewrite opportunities are compatible under a common ISA.

Figure 9 illustrates the area–latency design space under different limits on the number of ASIP variants. When the system is constrained to very few ASIPs (e.g., `num-chip=1` or `2`), the unified ISA must contain almost the full union of all instruction types used across the workloads. In such cases, the design has almost no flexibility to balance area against performance: hardware pruning opportunities are minimal, and even modest reductions in the ISA can lead to noticeable latency degradation.

As `num-chip` increases, each ISA subset becomes increasingly specialized, enabling more aggressive hardware trimming and significant area reductions. Remarkably, with only **three to six** ASIP variants, `æSIP` recovers the majority of the benefits achieved by full per-application specialization (`num-chip=10`).

A particularly attractive point appears at `num-chip=4`, where the system achieves a **14.17%** area reduction with only a **0.23%** latency increase (geometric mean). In comparison, even granting each program its own dedicated ASIP (`num-chip=22`) yields only a modest **19.11%** area reduction at the same latency target. This indicates that nearly all benefits of per-application specialization can already be captured with just a handful of ASIP variants. In other words, a small shared-ASIP pool delivers near-optimal hardware specialization while avoiding the substantial NRE overhead of designing, validating, and maintaining dozens of distinct ASIPs.

These observations are consistent with the empirical structure revealed in Fig. 8: although the workloads are diverse, they exhibit recurring patterns in instruction usage and rewrite behavior, which allow multiple programs to converge to compatible ISA subsets and share the same optimized ASIP design.

Overall, these results demonstrate that `æSIP` enables substantial ecosystem-level NRE amortization while preserving

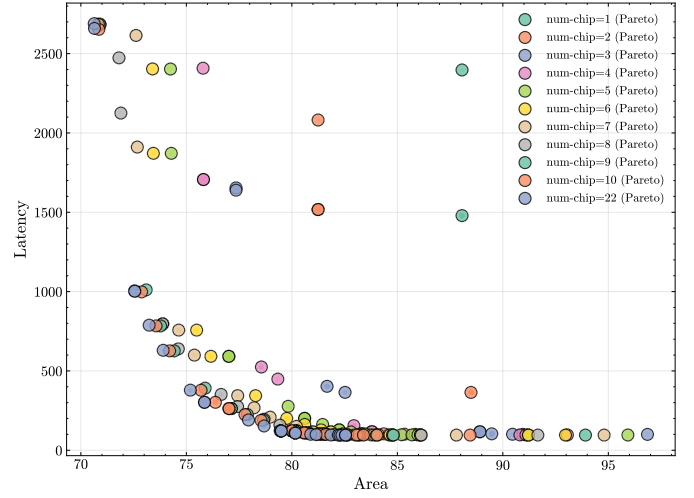


Fig. 9. Area–latency Pareto frontier under different maximum numbers of ASIP variants. Both axes report percentage values (normalized) but omit the percent sign for clarity. Even with only 3–6 ASIPs, `æSIP` achieves near-optimal specialization while greatly reducing NRE cost.

the benefits of specialization. The rewrite-driven ISA convergence not only makes multi-workload ASIP sharing feasible today, but also ensures that future workloads introduced over time can be rewritten to run on existing ASIPs, further extending the value of each design.

VI. CONCLUSION

The proliferation of resource-constrained embedded systems, from printed electronics to cryogenic quantum controllers, has created an urgent demand for extreme area and power optimization. While traditional ASIP design approaches rely on additive methodologies that expand instruction sets, and recent subtractive approaches like RISPs prune unused hardware, both fundamentally treat software as fixed constraints rather than optimization variables.

This paper introduces `æSIP`, the first hardware–software co-design framework that systematically explores the joint optimization space of program implementation and processor microarchitecture. By treating software as mutable through verified program rewriting, `æSIP` unlocks optimization opportunities invisible to prior hardware-only or software-only approaches.

Evaluation on MiBench and EmBench demonstrates that `æSIP` achieves up to 29.7% area reduction and 8.3% power savings compared to SOTA ASIP generator, while maintaining full functional correctness through end-to-end verification. For latency-critical applications, `æSIP` generates Pareto-optimal designs that expose the fundamental trade-offs between area efficiency and timing constraints. The ecosystem-level analysis further demonstrates how strategic ASIP sharing can substantially reduce NRE costs while preserving most specialization benefits, making extreme customization economically viable even for modest production volumes.

Upon publication, we will release `æSIP` as an open-source tool to enable further research and open-source chip design.

REFERENCES

- [1] A. Abate, C. David, P. Kesseli, D. Kroening, and E. Polgreen, “Counterexample guided inductive synthesis modulo theories,” in *International Conference on Computer Aided Verification*. Springer, 2018, pp. 270–288.
- [2] T. Achterberg, “What’s new in gurobi 9.0,” *Webinar Talk url: <https://www.gurobi.com/wp-content/uploads/2019/12/Gurobi-90-Overview-Webinar-Slides-1.pdf>*, vol. 5, no. 9, pp. 97–113, 2019.
- [3] T. Akiba, K. Imajo, H. Iwami, Y. Iwata, T. Kataoka, N. Takahashi, M. Moskal, and N. Swamy, “Calibrating research in program synthesis using 72,000 hours of programmer time,” *MSR, Redmond, WA, USA, Tech. Rep.*, 2013.
- [4] H. An, S. R. Nason-Tomaszewski, J. Lim, K. Kwon, M. S. Willsey, P. G. Patil, H.-S. Kim, D. Sylvester, C. A. Chestek, and D. Blaauw, “A power-efficient brain-machine interface system with a sub-mw feature extraction and decoding asic demonstrated in nonhuman primates,” *IEEE transactions on biomedical circuits and systems*, vol. 16, no. 3, pp. 395–408, 2022.
- [5] K. Atasu, L. Pozzi, and P. Ienne, “Automatic application-specific instruction-set extensions under microarchitectural constraints,” in *Proceedings of the 40th annual Design Automation Conference*, 2003, pp. 256–261.
- [6] C. Banbury, C. Zhou, I. Fedorov, R. Matas, U. Thakker, D. Gope, V. Janapa Reddi, M. Mattina, and P. Whatmough, “Micronets: Neural network architectures for deploying tinyml applications on commodity microcontrollers,” *Proceedings of machine learning and systems*, vol. 3, pp. 517–532, 2021.
- [7] J. Bardin, E. Jeffrey, E. Lucero, T. Huang, S. Das, D. Sank, O. Naaman, A. Megrant, R. Barends, T. White, M. Giustina, K. Satzinger, K. Arya, P. Roushan, B. Chiaro, J. Kelly, Z. Chen, B. Burkett, Y. Chen, A. Dunsworth, A. Fowler, B. Foxen, C. M. Gidney, R. Graff, P. Klimov, J. Mutus, M. McEwen, M. Neeley, C. Neill, C. Quintana, A. Vainsencher, H. Neven, and J. Martinis, “Design and characterization of a 28-nm bulk-cmos cryogenic quantum controller dissipating less than 2 mw at 3 k,” *IEEE Journal of Solid State Circuits*, vol. 54(11), pp. 3043 – 3060, 2019.
- [8] N. Bleier, J. Sartori, and R. Kumar, “Property-driven automatic generation of reduced-isa hardware,” in *2021 58th ACM/IEEE Design Automation Conference, DAC 2021*, ser. Proceedings - Design Automation Conference. United States: Institute of Electrical and Electronics Engineers Inc., Dec. 2021, pp. 349–354, publisher Copyright: © 2021 IEEE.; 58th ACM/IEEE Design Automation Conference, DAC 2021 ; Conference date: 05-12-2021 Through 09-12-2021.
- [9] N. Bleier, C. Lee, F. Rodriguez, A. Sou, S. White, and R. Kumar, “Flexicores: low footprint, high yield, field reprogrammable flexible microprocessors,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 831–846.
- [10] N. Bleier, M. H. Mubarik, F. Rasheed, J. Aghassi-Hagmann, M. B. Tahoori, and R. Kumar, “Printed microprocessors,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 213–226.
- [11] Brand, “Redundancy and don’t cares in logic synthesis,” *IEEE Transactions on Computers*, vol. C-32, no. 10, pp. 947–952, 1983.
- [12] C. Chen, G. Hu, C. Yu, Y. Ma, and H. Zhang, “E-morphic: Scalable equality saturation for structural exploration in logic synthesis,” *arXiv preprint arXiv:2504.11574*, 2025.
- [13] C. Chen, G. Hu, D. Zuo, C. Yu, Y. Ma, and H. Zhang, “E-syn: E-graph rewriting with technology-aware cost functions for logic synthesis,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [14] J. Cheng, S. Coward, L. Chelini, R. Barbalho, and T. Drane, “Seer: Super-optimization explorer for high-level synthesis using e-graph rewriting,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1029–1044. [Online]. Available: <https://doi.org/10.1145/3620665.3640392>
- [15] J. Cherry and W. Snyder, “OpenSTA: Static timing analyzer,” <https://github.com/The-OpenROAD-Project/OpenSTA>, 2020, version 2.2.0.
- [16] H. Cherupalli, H. Duwe, W. Ye, R. Kumar, and J. Sartori, “Bespoke processors for applications with ultra-low area and power constraints,” in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 41–54.
- [17] S. Coward, G. A. Constantinides, and T. Drane, “Abstract interpretation on e-graphs,” *arXiv preprint arXiv:2203.09191*, 2022.
- [18] —, “Automatic datapath optimization using e-graphs,” in *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*. IEEE, 2022, pp. 43–50.
- [19] S. Coward, T. Drane, E. Morini, and G. A. Constantinides, “Combining power and arithmetic optimization via datapath rewriting,” in *2024 IEEE 31st Symposium on Computer Arithmetic (ARITH)*. IEEE, 2024, pp. 24–31.
- [20] D. Das Sharma, G. Pasdast, Z. Qian, and K. Aygun, “Universal chiplet interconnect express (ucie): An open industry standard for innovations with chiplets at package level,” *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 12, no. 9, pp. 1423–1431, 2022.
- [21] L. De Moura and N. Björner, “Z3: An efficient smt solver,” in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2008, pp. 337–340.
- [22] H. Esmailzadeh, E. Blem, R. S. Amant, K. Sankaralingam, and D. Burger, “Dark silicon and the end of multicore scaling,” in *2011 38th Annual International Symposium on Computer Architecture (ISCA)*, 2011, pp. 365–376.
- [23] M. Felleisen, R. B. Findler, M. Flatt, S. Krishnamurthi, E. Barzilay, J. McCarthy, and S. Tobin-Hochstadt, “A programmable programming language,” *Commun. ACM*, vol. 61, no. 3, p. 62–71, Feb. 2018. [Online]. Available: <https://doi.org/10.1145/3127323>
- [24] Y. Feng and K. Ma, “Chiplet actuary: a quantitative cost model and multi-chiplet architecture exploration,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 121–126. [Online]. Available: <https://doi.org/10.1145/3489517.3530428>
- [25] R. E. Gonzalez, “Xtensa: A configurable and extensible processor,” *IEEE micro*, vol. 20, no. 2, pp. 60–70, 2000.
- [26] M. Gries and K. Keutzer, *Building ASIPs: The Mescal Methodology*. Berlin, Heidelberg: Springer-Verlag, 2005.
- [27] V. GROUP *et al.*, “Abc: a system for sequential synthesis and verification, release 70930,” 2007.
- [28] D. Große, L. Klemmer, and D. Bonora, “Using formal verification methods for optimization of circuits under external constraints,” in *2024 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2024, pp. 1–6.
- [29] M. Guthaus, J. Ringenberg, D. Ernst, T. Austin, T. Mudge, and R. Brown, “Mibench: A free, commercially representative embedded benchmark suite,” in *Proceedings of the Fourth Annual IEEE International Workshop on Workload Characterization. WWC-4 (Cat. No.01EX538)*, 2001, pp. 3–14.
- [30] K. Heljanko and I. Niemelä, “Bounded ltl model checking with stable models,” *Theory and Practice of Logic Programming*, vol. 3, no. 4-5, pp. 519–550, 2003.
- [31] P. Ienne and R. Leupers, *Customizable Embedded Processors: Design Technologies and Applications*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2006.
- [32] H. Jin, J. Yang, Y. Liu, B. Lyu, K. Zhang, and N. Bleier, “Mozart: A chiplet ecosystem-accelerator codesign framework for composable bespoke application specific integrated circuits,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.08873>
- [33] J. Kim, “Program synthesis is σ_0^0 -complete,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.16997>
- [34] S.-Y. Lee, H. Riener, and G. De Micheli, *External Don’t Cares in Logic Synthesis*. Cham: Springer International Publishing, 2023, pp. 33–47. [Online]. Available: https://doi.org/10.1007/978-3-031-28916-3_3
- [35] Z. Li, Z. Li, W. Tang, X. Zhang, Y. Yao, X. Si, F. Yang, K. Yang, and X. Ma, “Proving olympiad inequalities by synergizing llms and symbolic reasoning,” *arXiv preprint arXiv:2502.13834*, 2025.
- [36] N. P. Lopes, D. Menendez, S. Nagarakatte, and J. Regehr, “Provably correct peephole optimizations with alive,” in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI ’15. New York, NY, USA: Association for Computing Machinery, 2015, p. 22–32. [Online]. Available: <https://doi.org/10.1145/2737924.2737965>
- [37] H. Meyr, “Application specific instruction-set processors (asip’s) for wireless communications: design, cost, and energy efficiency vs. flexibility,” in *2004 International Symposium on System-on-Chip, 2004. Proceedings*. IEEE, 2004, pp. 1–2.

- [38] A. Mishchenko, R. Brayton, J.-H. R. Jiang, and S. Jang, "Scalable don't-care-based logic optimization and resynthesis," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 4, no. 4, Dec. 2011. [Online]. Available: <https://doi.org/10.1145/2068716.2068720>
- [39] A. Mishchenko, M. Case, R. Brayton, and S. Jang, "Scalable and scalably-verifiable sequential synthesis," in *2008 IEEE/ACM International Conference on Computer-Aided Design*, 2008, pp. 234–241.
- [40] M. H. Mubarik, D. D. Weller, N. Bleier, M. Tomei, J. Aghassi-Hagmann, M. B. Tahoori, and R. Kumar, "Printed machine learning classifiers," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 73–87.
- [41] A. Novikov, N. Vü, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehrabian *et al.*, "Alphaevolve: A coding agent for scientific and algorithmic discovery," *arXiv preprint arXiv:2506.13131*, 2025.
- [42] P. Panchekha, A. Sanchez-Stern, J. R. Wilcox, and Z. Tatlock, "Automatically improving accuracy for floating point expressions," *Acm Sigplan Notices*, vol. 50, no. 6, pp. 1–11, 2015.
- [43] E. Parisotto, A.-r. Mohamed, R. Singh, L. Li, D. Zhou, and P. Kohli, "Neuro-symbolic program synthesis," *arXiv preprint arXiv:1611.01855*, 2016.
- [44] D. Patterson, J. Bennett, M. Bennett, H. Chelin, D. Harris, J. Hellar, W. Jones, K. Moron, P. Savini, R. Shepherd, R. Simar, Z. Susskind, and S. Wallentowitz, "Embentech 2.0 and dsp 1.0: Modern embedded computing benchmarks," *Computer*, vol. 58, no. 5, pp. 37–47, 2025.
- [45] P. M. Phothilimthana, A. Thakur, R. Bodik, and D. Dhurjati, "Greenthumb: superoptimizer construction framework," in *Proceedings of the 25th International Conference on Compiler Construction*, ser. CC '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 261–262. [Online]. Available: <https://doi.org/10.1145/2892208.2892233>
- [46] M. Poletto and V. Sarkar, "Linear scan register allocation," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 21, no. 5, pp. 895–913, 1999.
- [47] A. Raisiardi, K. Iordanou, J. Kufel, K. Gudimetla, K. Myny, and E. Ozer, "Flexing risc-v instruction subset processors to extreme edge," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1147–1159. [Online]. Available: <https://doi.org/10.1145/3725843.3756036>
- [48] E. Rezunov, N. Zurstraßen, L. M. Reimann, and R. Leupers, "Automatic microarchitecture-aware custom instruction design for risc-v processors," 2025. [Online]. Available: <https://arxiv.org/abs/2509.15782>
- [49] RISC-V International, "RISC-V GNU Toolchain," <https://github.com/riscv-collab/riscv-gnu-toolchain>, 2025, accessed: 2025-08-15.
- [50] B. Saiki, J. Brough, J. Regehr, J. Ponce, V. Pradeep, A. Akhileshwaran, Z. Tatlock, and P. Panchekha, "Target-aware implementation of real expressions," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1069–1083. [Online]. Available: <https://doi.org/10.1145/3669940.3707277>
- [51] H. Savoj and R. Brayton, "The use of observability and external don't cares for the simplification of multi-level networks," in *27th ACM/IEEE Design Automation Conference*, 1990, pp. 297–301.
- [52] E. Schkufza, R. Sharma, and A. Aiken, "Stochastic superoptimization," in *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 305–316. [Online]. Available: <https://doi.org/10.1145/2451116.2451150>
- [53] S. Sethumurugan, S. Hegde, H. Cherupalli, and J. Sartori, "A scalable symbolic simulation tool for low power embedded systems," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 175–180. [Online]. Available: <https://doi.org/10.1145/3489517.3530433>
- [54] Q. Si, P. Chowdhury, R. Sreekumar, and B. C. Schafer, "Application Specific Approximate Behavioral Processor," *IEEE Transactions on Sustainable Computing*, vol. 8, no. 02, pp. 165–179, Apr. 2023. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/TSUSC.2022.3222117>
- [55] K. Skadron, M. R. Stan, W. Huang, S. Velusamy, K. Sankaranarayanan, and D. Tarjan, "Temperature-aware microarchitecture," *ACM SIGARCH Computer Architecture News*, vol. 31, no. 2, pp. 2–13, 2003.
- [56] SkyWater Technology and Google, Inc., "SkyWater SKY130 Open Source PDK," <https://github.com/google/skywater-pdk>, 2020, accessed: 2025-11-01.
- [57] A. Smith, G. H. Loh, M. J. Schulte, M. Ignatowski, S. Naffziger, M. Mantor, M. F. N. Kalyanasundharam, V. Alla, N. Malaya, J. L. Greathouse, E. Chapman, and R. Swaminathan, "Realizing the amd exascale heterogeneous processor vision : Industry product," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 876–889.
- [58] A. Solar-Lezama, "The sketching approach to program synthesis," in *Proceedings of the 7th Asian Symposium on Programming Languages and Systems*, ser. APLAS '09. Berlin, Heidelberg: Springer-Verlag, 2009, p. 4–13. [Online]. Available: https://doi.org/10.1007/978-3-642-10672-9_3
- [59] Synopsys, Inc., "Fusion Compiler: Rtl-to-gdsii digital design implementation system," <https://www.synopsys.com/implementation-and-signoff/fusion-compiler.html>, 2023, version: 2022.03-SP3, Accessed: 2025-11-01.
- [60] S. G. Sørensen, C. Bartsch, D. Stoffel, and W. Kunz, "Generation of formal cpu profiles for embedded systems," in *2022 IFIP/IEEE 30th International Conference on Very Large Scale Integration (VLSI-SoC)*, 2022, pp. 1–6.
- [61] M. B. Taylor, "Is dark silicon useful? harnessing the four horsemen of the coming dark silicon apocalypse," in *Proceedings of the 49th Annual Design Automation Conference*, ser. DAC '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 1131–1136. [Online]. Available: <https://doi.org/10.1145/2228360.2228567>
- [62] M. B. Taylor, L. Vega, M. Khazraee, I. Magaki, S. Davidson, and D. Richmond, "Asic clouds: specializing the datacenter for planet-scale applications," *Commun. ACM*, vol. 63, no. 7, p. 103–109, Jun. 2020. [Online]. Available: <https://doi.org/10.1145/3399734>
- [63] E. Torlak and R. Bodik, "Growing solver-aided languages with rosette," in *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software*, ser. Onward! 2013. New York, NY, USA: Association for Computing Machinery, 2013, p. 135–152. [Online]. Available: <https://doi.org/10.1145/2509578.2509586>
- [64] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong, "Solving olympiad geometry without human demonstrations," *Nature*, vol. 625, no. 7995, pp. 476–482, 2024.
- [65] E. Ustun, I. San, J. Yin, C. Yu, and Z. Zhang, "Impress: Large integer multiplication expression rewriting for fpga hls," in *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2022, pp. 1–10.
- [66] J. Van Praet, G. Goossens, D. Lanneer, and H. De Man, "Instruction set definition and instruction selection for asips," in *Proceedings of 7th International Symposium on High-Level Synthesis*. IEEE, 1994, pp. 11–16.
- [67] Z. Wan, C.-K. Liu, H. Yang, R. Raj, C. Li, H. You, Y. Fu, C. A. Wan, S. Li, Y. Kim *et al.*, "Towards efficient neuro-symbolic ai: From workload characterization to hardware architecture," *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, vol. 1, no. 1, pp. 53–68, 2024.
- [68] A. Weimerskirch and C. Paar, "Generalizations of the karatsuba algorithm for efficient implementations," *Cryptology ePrint Archive*, 2006.
- [69] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha, "egg: Fast and extensible equality saturation," *Proc. ACM Program. Lang.*, vol. 5, no. POPL, Jan. 2021. [Online]. Available: <https://doi.org/10.1145/3434304>
- [70] S. Xu and B. C. Schafer, "Approximating behavioral hw accelerators through selective partial extractions onto synthesizable predictive models," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019, pp. 1–8.
- [71] J. Yin, Z. Song, C. Chen, Q. Hu, and C. Yu, "Boole: Exact symbolic reasoning via boolean equality saturation," *arXiv preprint arXiv:2504.05577*, 2025.
- [72] A.-E.-A. Zayed and C. Dubach, "Dialegg: Dialect-agnostic mlir optimizer using equality saturation with egglog," in *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, ser. CGO '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 271–283. [Online]. Available: <https://doi.org/10.1145/3696443.3708957>

- [73] Y. Zhang, Y. R. Wang, O. Flatt, D. Cao, P. Zucker, E. Rosenthal, Z. Tatlock, and M. Willsey, “Better together: Unifying datalog and equality saturation,” *Proc. ACM Program. Lang.*, vol. 7, no. PLDI, Jun. 2023. [Online]. Available: <https://doi.org/10.1145/3591239>